

Технологии Программирования

Мегаструктуры

Вопрос!

Что есть программа с точки зрения компьютера

Перенос данных из одного блока памяти в другой с преобразованием их вида.

Многоуровневая архитектура

Память:

- Постоянная: SSD/HDD
- Оперативная: DRAM/NANO(?)
- Кэш: SRAM L3 - L2 - L1 - L0

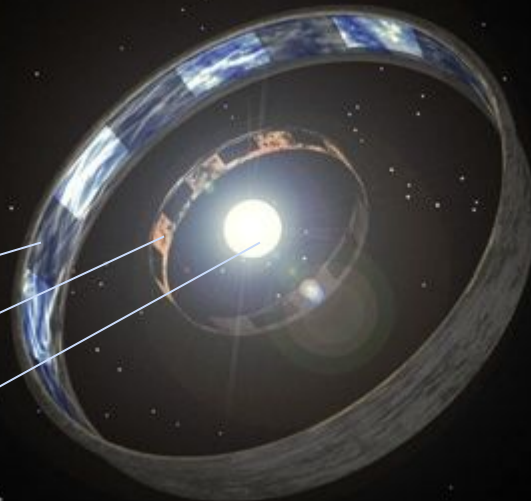
Характеристика:

- Малый размер
- Следующий уровень меньше предыдущего: *Core i9 Gen14: 36M->2M/c->80K/c*
- У каждого ядра multicore свой набор кэшей

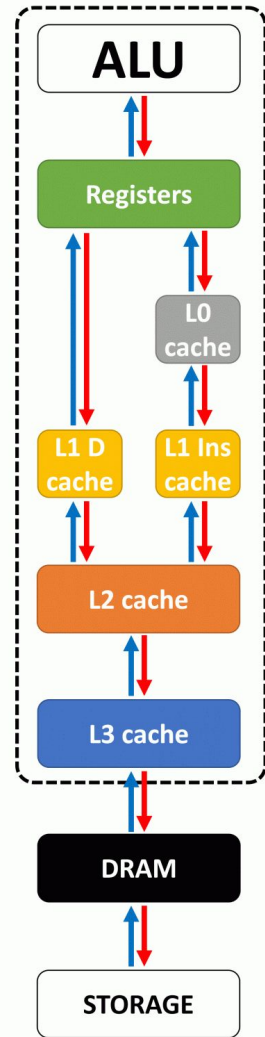
Описание:

- Обеспечение полной “занятости” процессора
- Быстрый доступ к данным и командам
- Когерентность, если несколько ядер содержит кэшей для каждого ядра идентично.

Самый эффективный алгоритм тот, что лезет в кэш cache friendly



JAG 1999



Система орбит

Характеристика:

- Описание сложности алгоритма
- $O(\text{сложность})$
- Всегда описывается худший случай.

Описание:

- Характеризует количество операций, необходимых для выполнения алгоритма
- Примеры:
 - Доступ к элементу массива произвольной длины по индексу $O(1)$
 - Сумма всех элементов одномерного массива длины n $O(n)$,
 - Бинарный поиск в отсортированном массиве $O(\log_2 N)$,
 - Сумма всех элементов 2D матрицы $O(n^2)$

Мегаструктуры

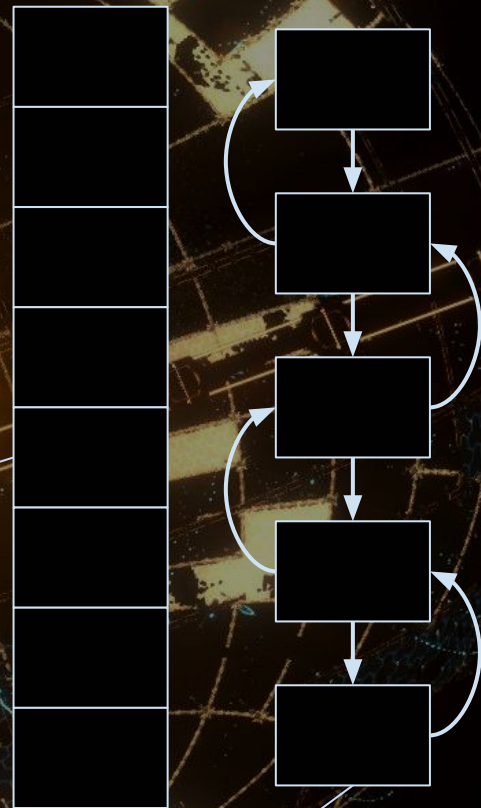
Общая характеристика:

Эффективная работа программы зависит от правильно выбранной структуры в которой хранятся данные для работы программы.

Выбор структуры зависит от алгоритма, который реализуется в программе, и требованиям доступа к данным.

Виды:

- Массив
- Список
- Дерево
- Стэк
- Хэш Таблицы (не путать с Кэшем)



Линейная структура

Динамический Массив

Тип STL: `std::vector<type>`

Характеристика:

- Автоматическое расширение
- Добавление в произвольную точку по индексу
- Удаление дыр

Статический Массив

Тип STL: `std::array<type,size>`

Характеристика:

- Фиксированный размер на этапе компиляции
- Полный аналог C-массива

Описание:

Обеспечивает непрерывный блок памяти для хранения данных с доступом по индексу $O(1)$. Подходит для алгоритмов где важна скорость доступа к произвольному элементу и мало добавлений/удалений, или количество данных фиксировано.

Лифт в деталях

Характеристики:

- Доступ к произвольной ячейке: $O(1)$
- Добавление: $O(1)$ но $O(n)$
 - полное копирование содержимого массива в новое место
- Удаление: $O(n)$
 - полное копирование
- Требование: непрерывность индексации

Описание:

Непрерывный блок памяти. Удобен для быстрого доступа к элементам по индексам, когда random access важен или важна скорость работы с памятью (cache friendly).

0
1
2
3
4
5
6
7

Для эффективной работы с vector важно следить за 2мя параметрами - size и capacity.

Порталы

Двусвязный список

Тип STL: `std::list<type>`

Характеристики:

- Быстрое добавление элементов в начало и конец.
- Быстрое добавление элементов в середину (по итератору)
- Доступ к элементам последовательно с 2х сторон

Односвязный список FIFO

Тип STL: `std::forward_list<type>`

Характеристики:

- Быстрое добавление элементов в начало.
- Быстрое добавление элементов в середину (по итератору)
- Доступ к элементам последовательно с начала
- Компактнее чем `list`

Описание:

Обеспечивает быстрое добавление и удаление элементов из любой точки списка. Подходит для алгоритмов где важна скорость добавления элементов “на лету”, и где доступ к ним осуществляется строго последовательно.

Порталы в деталях

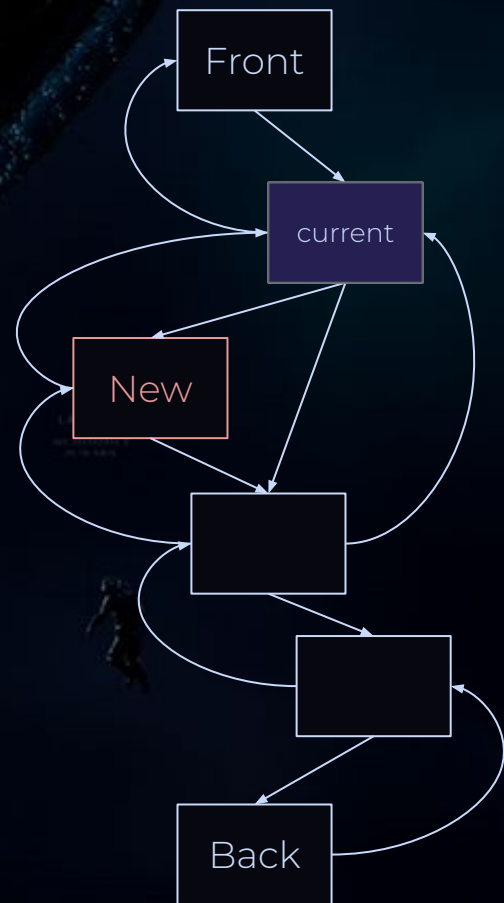
Характеристики:

- Доступ к произвольной ячейке: $O(n)$
- Добавление: $O(1)$
- Поиск: $O(n)$
- Удаление: $O(n)$

Описание:

Когда скорость доступа к элементам не важна, важно обходить все элементы по порядку. Наиболее эффективной будет вставка элемента в ходе итерирования по списку, если нет, то сложность вставки будет включать поиск, а это уже $O(n)$.

В памяти каждый элемент списка лежит в произвольном месте, поэтому итерирование по нему не эффективно с точки зрения Кэша.



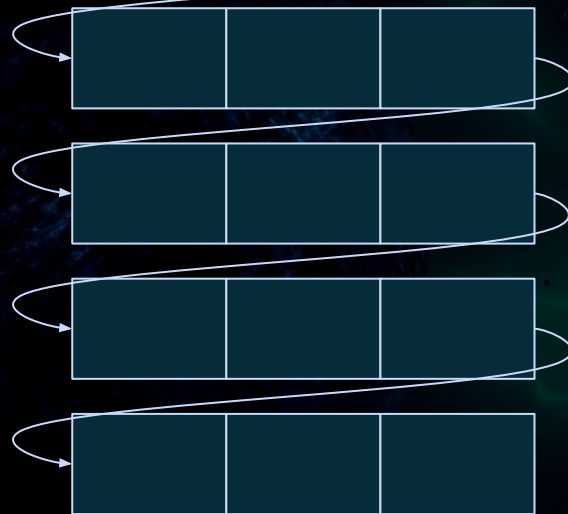
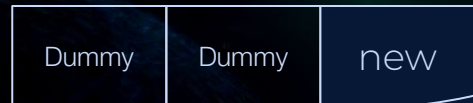
Дека

Характеристики:

- Доступ к произвольной ячейке: $O(1)$
- Добавление и удаление в начало/конец: $O(1)$
- Поиск: $O(n)$
- Добавление/Удаление: $O(n)$

Описание:

Когда скорость доступа к элементам важна, также важна скорость добавления элементов в начало и конец. Отличается от списка тем что состоит из связанного набора массивов. При добавлении элементов в начало или конец не требует копирования всего массива данных



Экран

Тип STL: `std::stack<type>`

Характеристики:

- Быстрое добавление элементов в начало.
- Доступ только к первому элементу.

Описание:

Обеспечивает быстрое добавление элементов в начало/вершину стека и доступ только к вершине.

Подходит для алгоритмов работающих с текущим - независимым состоянием, и также важно иметь возможность вернуться к предыдущим сохраненным состояниям.

Экран в деталях

Характеристики:

- Архитектура: LiFo - Last In First Out
- Доступ к ячейке: $O(1)$
- Добавление: $O(1)$
- Удаление: $O(1)$
- Требование: только первый элемент

Описание:

После добавления элемента он становится первым, замещая предыдущий. После его удаления возвращает предыдущий сохраненный элемент.

Когда скорость доступа к элементам не важна, важно иметь возможность работать с текущим элементом и откатиться к предыдущему когда работа с текущим закончена.

new

Дерево “Дайсона”

Структура:

Бинарное дерево, сбалансированное. Красно-чёрное дерево.

Характеристика:

- Быстрое добавление элементов в список
- Нельзя контролировать положение куда будет добавлен элемент, положение будет определено самой структурой
- Быстрый доступ к произвольному элементу по ключу

Описание:

Обеспечивает быстрое добавление элементов и быстрый поиск элементов внутри контейнера. Подходит для алгоритмов требующих работу со словарями, когда необходимо проверить наличие элемента в контейнере (set) или по ключу получить связанные с ним данные (map). Эффективен для строк.

Тип STL: `std::map<key, value>`
`std::set<key>`

Дерево: в деталях

Характеристики:

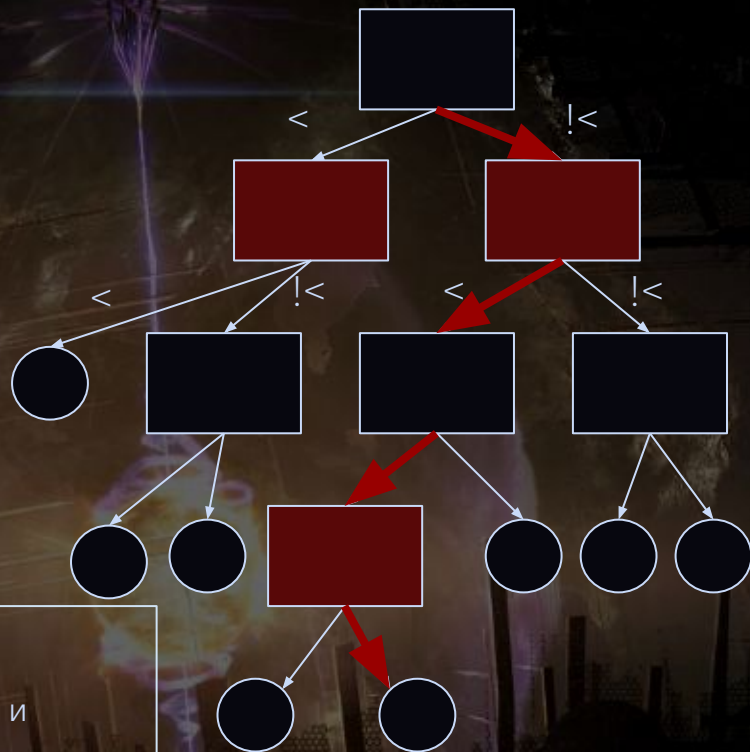
- Доступ к ячейке: $O(\log_2 N)$
- Добавление: $O(\log_2 N)$
- Удаление: $O(\log_2 N)$
- Требование: оператор $<$

Описание:

Высокая скорость доступа к элементам. Высокая скорость добавления и удаления элемента. Подходит для поиска.

Сложность реализации, для эффективности и обеспечения $\log N$, дерево должно балансироваться (стараться держать одну глубину для всех значений).

Ключ не является вычисляемым, тип ключа должен обладать оператором “меньше”.



Системы хаоса

Характеристики:

- Доступ к ячейке: $O(1)$ но $O(m)$
- Добавление: $O(1)$
- Удаление: $O(1)$ но $O(m)$
- Требование: hash функция
- Особенность: конфликты

Тип STL:

`std::unordered_map<key, value>`

`std::unordered_set<key>`

Описание:

Ключ вычисляется с помощью hash функции. Однозначно определяет положение в структуре. Эффективность сильно зависит от выбора функции, при неверном выборе функции возможны конфликты. Для своего типа данных требуется своя функция. Плохо работает со строками.



Системы хаоса

Пример: текстовый хэш

Данные: “а”, “столб”, “стена”, “мы”, “дверь”, “лампа”

Функция: $F(\text{слова}) = N$ букв

Результат: Конфликты на элементе 5.

Поиск не эффективен.





THANK YOU!